

Giriş ve Genel Bakış

B ve B+ Ağaçları Nedir?

B-Ağaçları (B-Trees):1970'lerde Rudolf Bayer ve Edward M. McCreight tarafından geliştirilen, kendi kendini dengeleyen (self-balancing) ve ikili arama ağaçlarının genişletilmiş bir versiyonu olan veri yapılarıdır.

B+ Ağaçları (B+ Trees): B-Ağacının modern sistemler için optimize edilmiş halidir. En temel farkı, tüm anahtarların ve verilerin sadece en alt katmanda (yaprak düğümlerde) saklanmasıdır.

Temel Amaç: Disk gibi yavaş çalışan depolama birimlerinde, minimum okuma/yazma (I/O) işlemiyle milyarlarca veri arasından aranılan kaydı nanosaniyeler içinde bulmaktır.

Veri Tabanlarında Kullanımı

Veri tabanları, performanslarını doğrudan disk erişim hızına borçludur. B/B+ Ağaçları bu performansın kalbidir.

MySQL (InnoDB Motoru):

MySQL'in varsayılan depolama motoru olan InnoDB, B+ Ağacı yapısını kullanır.

Clustered Index (Kümelenmiş İndeks): Tablonun birincil anahtarı (Primary Key) bir B+ Ağacı olarak modellenir. Verinin kendisi doğrudan en alt katmandaki yapraklarda saklanır. Bir ID arandığında, veri diske sadece birkaç kez dokunularak doğrudan çekilir.

PostgreSQL:

Varsayılan ve en çok tercih edilen indeksleme türü B-Tree'dir.

PostgreSQL'in B-Tree tasarımı, yüksek eşzamanlılık (concurrency) altında kilitleme (locking) mekanizmalarını minimuma indirmek için optimize edilmiştir. Aynı anda binlerce kişi veri okuyup yazarken sistemin kilitlenmesini engeller.

MongoDB:

Doküman tabanlı (NoSQL) bir veri tabanı olmasına rağmen, arka plandaki "WiredTiger" depolama motoru indeksleme için B-Tree türevlerini kullanır.

Büyük JSON dokümanlarının konumlarını hızlıca bulmak için bu ağaç yapısından yararlanır.

Bilgisayarınızda bir klasörü açtığınızda veya bir dosyayı arattığınızda, işletim sistemi arka planda bu ağaçları yürütür.

Dosya Sistemlerinde Kullanımı

NTFS (Windows): Windows'un kullandığı bu dosya sistemi, büyük klasörlerdeki dosya isimlerini ve bu dosyaların diskteki fiziksel konumlarını (cluster adreslerini) B+ Ağaçları ile indeksler. Klasörde 100.000 dosya olsa bile aradığınız dosya anında açılır.

ext4 (Linux): Linux dünyasının popüler dosya sistemi, çok büyük dizinleri (büyük miktarda alt klasör içeren yapıları) yönetmek için HTree adı verilen, B-Tree tabanlı özel bir hashing ve ağaç yapısı kullanır.

APFS (Apple File System): macOS, iOS ve watchOS cihazlarda kullanılan modern dosya sistemi, depolama alanını verimli yönetmek, hızlı dosya klonlama ve anlık görüntü (snapshot) almak için B+ Ağaçlarını merkeze koyar.

Arama Motorları ve Büyük Veri Sistemleri

Arama Motorları

(Ters Çevrilmiş İndeks – Inverted Index):

Google veya DuckDuckGo gibi arama motorları internetteki kelimeleri indeksler.

Bir kelimenin hangi web sitelerinde geçtiğinin listesini tutan veri yapılarında hızlı arama ve kelime aralığı tarama işlemleri için B+ Ağaçları kritik rol oynar.

Büyük Veri (Big Data) Depoları:

Apache Cassandra veya HBase gibi sistemler LSM-Tree (Log-Structured Merge-tree) kullansa da, bu sistemlerin içindeki disk bloklarının (SSTable) kendi içindeki indeks haritaları oluşturulurken yine

B-Tree mantığından ilham alınır. Disk üzerindeki devasa blokların yer tespiti bu ağaçlarla yapılır.

Neden Modern Sistemlerde Tercih Edilmektedir?

Modern mimarilerin bu ağaçları seçmesinin 4 teknik nedeni vardır:

- 1. Dengeli Ağaç Yapısı (Perfect Balance):** Ağacın tüm yaprak düğümleri aynı derinliktedir. Bu durum, veri seti ne kadar büyürse büyüsün, arama, ekleme ve silme işlemlerinin her zaman kararlı bir şekilde $O(\log n)$ zaman karmaşıklığıyla çalışmasını sağlar. Sürpriz yavaşlamalar yaşanmaz.
- 2. Yüksek Dallanma Faktörü (High Fan-out):** Standart bir ikili ağaç (Binary Tree) her düğümde 2'ye ayrılırken, B-Ağaçları tek bir düğümde yüzlerce (örneğin 512 veya 1024) çocuk düğüm bağlantısı tutabilir. Bu sayede milyarlarca veriden oluşan bir ağacın yüksekliği sadece 3 veya 4 katman olur. Sadece 4 adımda milyarlarca veri taranabilir.

3. Disk ve Sayfa Optimizasyonu (Page-Block Alignment):
Diskler veriyi bayt bayt değil, "sayfa" (page) adı verilen bloklar (genelde 4KB, 8KB veya 16KB) halinde okur. B-Ağacının tek bir düğümü, tam olarak bir disk bloğuna sığacak boyutta tasarlanır. Tek bir disk okuma işleminde (I/O) yüzlerce indeks bilgisi birden belleğe yüklenir.

4. B+ Ağacının Sıralı Erişim Gücü (Range Queries):
B+ Ağaçlarında tüm gerçek veriler en alt kattadır ve bu yaprak düğümler birbirine bağlı liste (linked list) ile sağa doğru bağlanmıştır. Bir aralık sorgusu yapıldığında (örneğin: "Fiyatı 100 TL ile 500 TL arasındaki ürünleri getir"), ağaçtan 100 bulunur ve yukarı katmanlara bir daha hiç çıkmadan yapraklar üzerinden sağa doğru yürünerek 500'e kadar olan tüm veriler tek seferde toplanır.

B+ Tree (B+ Ağacı) Tasarımı:

Kütüphane yönetimi sistemi değiştirir:

Giriş katında, birinci katta ve ikinci katta asla tek bir gerçek kitap bile tutulmaz. Üst katların tamamı sadece devasa ve detaylı "Yönlendirme Tabelalarından" ibarettir. Kitapların (verilerin) tamamı sadece ve sadece en alt katta (depoda) yan yana dizilmiştir.

Dahası, en alt kattaki kitap rafları arkadan raylı bir sistemle (bağlı liste) birbirine kesintisizce bağlanmıştır. Analoji: İndeksler üst katlarda, tüm gerçek veriler ise sadece en alt kattaki yaprak düğümlerde saklanır.

Somut Örnek ve Fayda Analizi (Kütüphane Üzerinden)

Senaryo: Kütüphaneden "Kemal Tahir'in yazmış olduğu tüm kitapları" topluca ödünç almak istiyorsunuz.

B-Tree Sisteminde Olsaydık:

Tabelalara bakarak Kemal Tahir'in ilk kitabını 2. katta bir masada bulursunuz. İkinci kitabını bulmak için tekrar üst katlardaki tabelalara çıkmalı, oradan başka bir koridora yönelmeli, yani katlar arasında sürekli aşağı yukarı mekik dokumalısınız.

Veri tabanındaki karşılığı: Disk kafasının sürekli ileri geri hareket etmesi (High Disk I/O) ve sistemin yavaşlamasıdır.

B+ Tree Sisteminde (Modern Tercih):

Üst katlardaki tabelaları hızla geçerek direkt en alt kata (depoya) iner ve Kemal Tahir'in ilk kitabını rafta bulursunuz.

Sistem B+ Tree olduğu için, o rafın hemen devamında Kemal Tahir'in diğer tüm kitapları sıralı olarak yan yana dizilmiştir. Üst katlara bir daha hiç uğramadan, reyonda sağa doğru yürüyerek tüm kitapları tek seferde sepetinize doldurursunuz.

Somut Faydası:

Nokta Atışı Arama: Tek bir veriyi ararken üst katlarda veri taşınmadığı için tabelalar daha büyüktür, bu sayede en alt kata çok daha az adımda (minimum disk okumasıyla) inilir.



Mükemmel Aralık Sorgusu: Ardışık veya belirli bir aralıktaki veriler isendiğinde, diske sadece bir kez konumlanılır ve veri doğrusal bir hat üzerinde kesintisizce okunur. Günümüz sistemlerinin en çok ihtiyaç duyduğu hız tam olarak budur.

* Bu veri yapıları neden önemlidir?

Çünkü B ve B+ ağaçları, milyarlarca satırlık devasa veriler içinde bile aradığımız şeyi diske en az kez dokunarak, sabit ve kararlı bir hızda bulmamızı sağlar.

* Büyük veri çağında neden ihtiyaç duyulmaktadır?

Büyük veri çağında veri sadece büyümüyor, aynı zamanda anlık olarak işlenmesi gerekiyor. Sosyal medya etkileşimleri, bankacılık işlemleri veya anlık arama motoru sorguları saniyede milyonlarca veri üretir. Eğer bu veriler düzgün indekslenmezse sistemler kilitlenir. B+ Ağaçları, özellikle 'belirli tarih aralığındaki verileri getir' gibi büyük verinin en çok ihtiyaç duyduğu aralık sorgularını disk üzerinde doğrusal ve kesintisiz yapabildiği için vazgeçilmezdir.

* Sizce gelecekte de kullanılmaya devam edecek midir?

Evet, kesinlikle devam edecek. Yapısal olarak bu ağaçlar disk mimarisine tam oturduğu için, geometrik bir veri artışı karşısında bile tahtını kolay kolay başka bir veri yapısına kaptırmayacaktır.

Teşekkürler!

Kaynak Sayfası

dev.mysql.com

postgresql.org

mongodb.com/docs/manual/indexes/

learn.microsoft.com

ext4.wiki.kernel.org

developer.apple.com

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. Acta Informatica, 1(3), 173-189.

